

반복문

1. 리스트와 반복문

시작하기 전에

- 리스트(list)

- 여러 가지 자료를 저장할 수 있는 자료
- 자료들을 모아서 사용할 수 있게 해 줌
- 대괄호 내부에 자료들 넣어 선언

```
>>> array = [273, 32, 103, "문자열", True, False]
>>> print(array)
[273, 32, 103, '문자열', True, False]
```

리스트 선언하고 요소에 접근하기

- 요소(element)

- 리스트의 대괄호 내부에 넣는 자료(쉼표로 구분하여 입력)

```
[요소, 요소, 요소...]
```

```
>>> [1, 2, 3, 4] # 숫자만으로 구성된 리스트
[1, 2, 3, 4]
>>> ["안", "녕", "하", "세", "요"] # 문자열만으로 구성된 리스트
['안', '녕', '하', '세', '요']
>>> [273, 32, 103, "문자열", True, False] # 여러 자료형으로 구성된 리스트
[273, 32, 103, '문자열', True, False]
```

리스트 선언하고 요소에 접근하기

- 리스트 내부의 요소 각각 사용하려면 리스트 이름 바로 뒤에 대괄호 입력 후 자료의 위치 나타내는 숫자 입력

```
list_a = [273, 32, 103, "문자열", True, False]
```

list_a	273	32	103	문자열	True	False
	[0]	[1]	[2]	[3]	[4]	[5]

- 인덱스(index)
 - 대괄호 안에 들어간 숫자

```
>>> list_a = [273, 32, 103, "문자열", True, False]
>>> list_a[0]
273
>>> list_a[1]
32
>>> list_a[2]
103
>>> list_a[1:3]
[32, 103]
```

➤ 결과로 [32, 103] 출력

리스트 선언하고 요소에 접근하기

- 리스트 특정 요소를 변경할 수 있음

```
>>> list_a = [273, 32, 103, "문자열", True, False]
>>> list_a[0] = "변경"
>>> list_a
['변경', 32, 103, '문자열', True, False]
```

[0] 번째 요소가 변경되었습니다.

list_a	↑					
	변경	32	103	문자열	True	False
	[0]	[1]	[2]	[3]	[4]	[5]

리스트 선언하고 요소에 접근하기

- 대괄호 안에 음수 넣어 뒤에서부터 요소 선택하기

```
>>> list_a = [273, 32, 103, "문자열", True, False]
>>> list_a[-1]
False
>>> list_a[-2]
True
>>> list_a[-3]
'문자열'
```

273	32	103	문자열	True	False
[-6]	[-5]	[-4]	[-3]	[-2]	[-1]

리스트 선언하고 요소에 접근하기

- 리스트 접근 연산자를 이중으로 사용할 수 있음

```
>>> list_a = [273, 32, 103, "문자열", True, False]
>>> list_a[3]
'문자열'
>>> list_a[3][0]
'문'
```

- 리스트 여러 개를 가지는 리스트

```
>>> list_a = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>>> list_a[1]
[4, 5, 6]
>>> list_a[1][1]
5
```


리스트 선언하고 요소에 접근하기

- 리스트에서의 IndexError 예외
 - 리스트의 길이 넘는 인덱스로 요소에 접근하려는 경우 발생

```
>>> list_a = [273, 32, 103]
>>> list_a[3]
```

 오류

```
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    IndexError: list index out of range
```

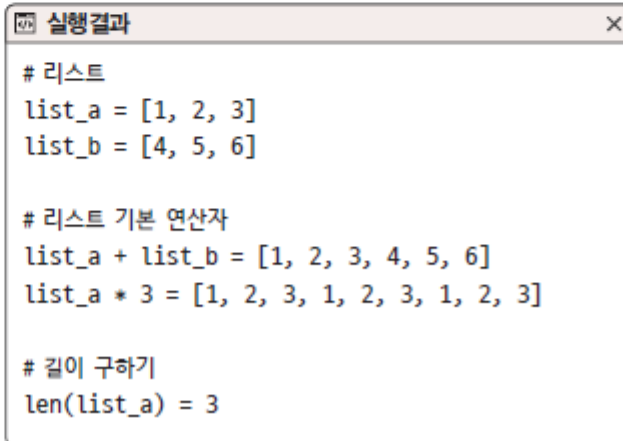
리스트 연산하기: 연결(+), 반복(*), len()

- 예시 – 리스트 연산자

```
01  # 리스트를 선언합니다.  
02  list_a = [1, 2, 3]  
03  list_b = [4, 5, 6]  
04  
05  # 출력합니다.  
06  print("# 리스트")  
07  print("list_a =", list_a)
```

리스트 연산하기: 연결(+), 반복(*), len()

```
08 print("list_b =", list_b)
09 print()
10
11 # 기본 연산자
12 print("# 리스트 기본 연산자")
13 print("list_a + list_b =", list_a + list_b)
14 print("list_a * 3 =", list_a * 3)
15 print()
16
17 # 함수
18 print("# 길이 구하기")
19 print("len(list_a) =", len(list_a))
```



실행결과

```
# 리스트
list_a = [1, 2, 3]
list_b = [4, 5, 6]

# 리스트 기본 연산자
list_a + list_b = [1, 2, 3, 4, 5, 6]
list_a * 3 = [1, 2, 3, 1, 2, 3, 1, 2, 3]

# 길이 구하기
len(list_a) = 3
```

리스트 연산하기: 연결(+), 반복(*), len()

- 13행에서 문자열 연결 연산자 사용해 2, 3행과 7, 8행에서 선언 및 출력된 list_a와 list_b의 자료 연결
- 14행에서 문자열 반복 연산자 사용해 list_a의 자료 3번 반복
- 19행에서 len() 함수로 list_a에 들어있는 요소의 개수 구함

리스트에 요소 추가하기: append(), insert()

- append() 함수

- 리스트 뒤에 요소를 추가

```
리스트명.append(요소)
```

- insert() 함수

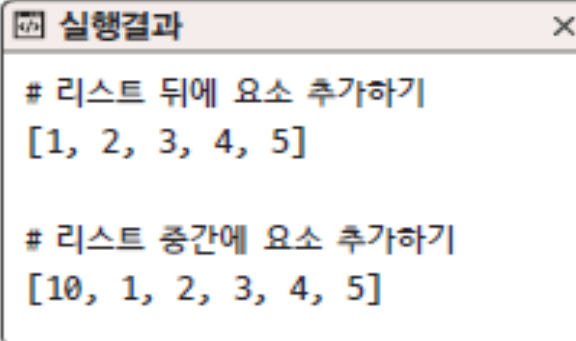
- 리스트 중간에 요소를 추가

```
리스트명.insert(위치, 요소)
```

리스트에 요소 추가하기: append(), insert()

- 예시 - 리스트에 요소 추가하기

```
01  # 리스트를 선언합니다.  
02  list_a = [1, 2, 3]  
03  
04  # 리스트 뒤에 요소 추가하기  
05  print("# 리스트 뒤에 요소 추가하기")  
06  list_a.append(4)  
07  list_a.append(5)  
08  print(list_a)  
09  print()  
10  
11  # 리스트 중간에 요소 추가하기  
12  print("# 리스트 중간에 요소 추가하기")  
13  list_a.insert(0, 10)  
14  print(list_a)
```



실행결과

```
# 리스트 뒤에 요소 추가하기  
[1, 2, 3, 4, 5]  
  
# 리스트 중간에 요소 추가하기  
[10, 1, 2, 3, 4, 5]
```

리스트에 요소 추가하기: append(), insert()

- 6 및 7행 실행 결과

```
list_a.append(4)
list_a.append(5)
```

1	2	3	4	5
[0]	[1]	[2]	[3]	[4]

- 13행 실행 결과

```
list_a.insert(0, 10)
```

삽입할 위치
삽입할 값

10	1	2	3	4	5
[0]	[1]	[2]	[3]	[4]	[5]

↓
0번째 위치에 10을 추가합니다.

리스트에 요소 추가하기: append(), insert()

- extend() 함수

- 원래 리스트 뒤에 새로운 리스트의 요소 모두 추가
- 매개변수로 리스트 입력

```
>>> list_a = [1, 2, 3]
>>> list_a.extend([4, 5, 6])
>>> print(list_a)
[1, 2, 3, 4, 5, 6]
```

→ 매개변수를 리스트 형태로 넣습니다.

리스트에 요소 추가하기: append(), insert()

- 리스트 연결 연산자와 요소 추가의 차이
 - 리스트 연결 연산자 사용하면 결과상 원본에 변화는 없음

```
>>> list_a = [1, 2, 3]
>>> list_b = [4, 5, 6]
>>> list_a + list_b → 리스트 연결 연산자로 연결하니,
[1, 2, 3, 4, 5, 6] → 실행 결과로 [1, 2, 3, 4, 5, 6]이 나왔습니다.
>>> list_a
[1, 2, 3] → list_a에는 어떠한 변화도 없습니다(비파괴적 처리).
>>> list_b
[4, 5, 6] → list_b에는 어떠한 변화도 없습니다(비파괴적 처리).
```

리스트에 요소 추가하기: append(), insert()

- extend() 함수 사용할 경우

```
>>> list_a = [1, 2, 3]
>>> list_b = [4, 5, 6]
>>> list_a.extend(list_b) → 실행결과로 아무 것도 출력하지 않았습니다.
>>> list_a → 앞에 입력했던 list_a 자체에 직접적인 변화가 있습니다(파괴적 처리).
[1, 2, 3, 4, 5, 6]
>>> list_b
[4, 5, 6]
```

- 파괴적 / 비파괴적

- 파괴적: append(), insert(), extend() 함수와 같이 리스트에 직접적인 영향을 주는 함수 또는 연산

리스트에 요소 제거하기

- 인덱스로 제거하기: del 키워드, pop() 함수

```
del 리스트명[인덱스]
```

```
리스트명.pop(인덱스)
```

- 예시 - 리스트 요소 하나 제거하기

```
01 list_a = [0, 1, 2, 3, 4, 5]
02 print("# 리스트의 요소 하나 제거하기")
03
04 # 제거 방법[1] - del
05 del list_a[1]
06 print("del list_a[1]:", list_a)
07
08 # 제거 방법[2] - pop()
09 list_a.pop(2)
10 print("pop(2):", list_a)
```

실행결과

```
# 리스트의 요소 하나 제거하기
del list_a[1]: [0, 2, 3, 4, 5]
pop(2): [0, 2, 4, 5]
```

리스트에 요소 제거하기

- 5행 실행하면 자료에서 1 제거

```
del list_a[1]
```

0	2	3	4	5
[0]	[1]	[2]	[3]	[4]

- 9행에서 2번째 요소인 3 제거

```
list_a.pop(2)
```

0	2	4	5
[0]	[1]	[2]	[3]

리스트에 요소 제거하기

- del 키워드 사용할 경우 범위 지정해 리스트 요소를 한꺼번에 제거 가능

```
>>> list_b = [0, 1, 2, 3, 4, 5, 6]
>>> del list_b[3:6]
>>> list_b
[0, 1, 2, 6]
```

- 범위 한 쪽을 입력하지 않으면 지정 위치 기준으로 한쪽을 전부 제거
 - [:3] - 3 왼쪽 제거 , [3:] - 3 오른쪽 제거

```
>>> list_c = [0, 1, 2, 3, 4, 5, 6]
>>> del list_c[:3]
>>> list_c
[3, 4, 5, 6]
```

리스트에 요소 제거하기

- 값으로 제거하기: remove() 함수
 - 특정 값을 지정하여 제거

```
리스트.remove(값)
```

```
>>> list_c = [1, 2, 1, 2]    # 리스트 선언하기
>>> list_c.remove(2)         # 리스트의 요소를 값으로 제거하기
>>> list_c
[1, 1, 2]
```

리스트에 요소 제거하기

- 모두 제거하기 : `clear()` 함수
 - 리스트 내부의 요소를 모두 제거

```
리스트.clear()
```

```
>>> list_d = [0, 1, 2, 3, 4, 5]
>>> list_d.clear()
>>> list_d
[] → 요소가 모두 제거되었습니다.
```

리스트 정렬하기: sort()

- 리스트 정렬 – 기본 오름차순

```
리스트.sort()
```

```
>>> list_e = [52, 273, 103, 32, 275, 1, 7]
```

```
>>> list_e.sort() # 오름차순 정렬
```

```
>>> list_e
```

```
[1, 7, 32, 52, 103, 273, 275]
```

```
>>> list_e.sort(reverse=True) # 내림차순 정렬
```

→ *이 함수 만들기: 키워드 매개변수"에서 자세하게 다루는 문법입니다.*

```
>>> list_e
```

```
[275, 273, 103, 52, 32, 7, 1]
```


리스트 내부에 있는지 확인하기 : in/not in 연산자

- in 연산자

- 특정 값이 리스트 내부에 있는지 확인

값 in 리스트

```
>>> list_a = [273, 32, 103, 57, 52]
>>> 273 in list_a
True
>>> 99 in list_a
False
>>> 100 in list_a
False
>>> 52 in list_a
True
```

리스트 내부에 있는지 확인하기 : in/not in 연산자

- not in 연산자

- 리스트 내부에 해당 값이 없는지 확인

```
>>> list_a = [273, 32, 103, 57, 52]
>>> 273 not in list_a
False
>>> 99 not in list_a
True
>>> 100 not in list_a
True
>>> 52 not in list_a
False
>>> not 273 in list_a
False
```

for 반복문

- 반복문

- 컴퓨터에 반복 작업을 지시

```
print("출력")  
print("출력")  
print("출력")  
print("출력")  
print("출력")
```

```
for i in range(100):  
    print("출력")
```

→ 반복에 사용할 수 있는 자료

for 반복문 : 리스트와 함께 사용하기

- 문자열, 리스트, 딕셔너리, 범위 등과 조합하여 for 반복문을 사용

for 반복자 in 반복할 수 있는 것:

코드

```
01  # 리스트를 선언합니다.  
02  array = [273, 32, 103, 57, 52]  
03  
04  # 리스트에 반복문을 적용합니다.  
05  for element in array:  
06      # 출력합니다.  
07      print(element)
```

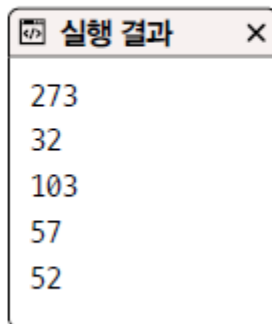
실행결과

273
32
103
57
52

for 반복문 : 리스트와 함께 사용하기

- 예시 - for 반복문과 리스트

```
01  # 리스트를 선언합니다.  
02  array = [273, 32, 103, 57, 52]  
03  
04  # 리스트에 반복문을 적용합니다.  
05  for element in array:  
06      # 출력합니다.  
07      print(element)
```



실행 결과

273
32
103
57
52

중첩 리스트와 중첩 반복문(누적 예제)

- 중첩 반복문은 일반적으로 n-차원 처리를 할 때 사용
 - 1차원 리스트 - [1, 2, 3]처럼 리스트가 한 겹으로 감싸진 리스트
 - 2차원 리스트 - [[1, 2, 3], [4, 5, 6]]처럼 두 겹으로 감싸진 리스트

1차원 리스트

1	2	3
---	---	---

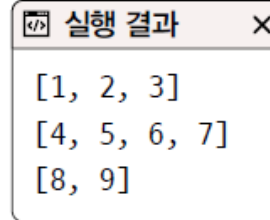
2차원 리스트

1	2	3
4	5	6

중첩 리스트와 중첩 반복문(누적 예제)

- 예시 - 2차원 리스트에 반복문 한번 사용하기

```
01 list_of_list = [  
02     [1, 2, 3],  
03     [4, 5, 6, 7],  
04     [8, 9]  
05 ]  
06  
07 for items in list_of_list:  
08     print(items)
```



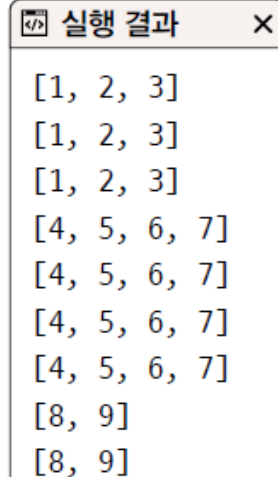
실행 결과

```
[1, 2, 3]  
[4, 5, 6, 7]  
[8, 9]
```

중첩 리스트와 중첩 반복문

- 예시 - 2차원 리스트에 반복문 두 번 사용하기

```
01 list_of_list = [  
02     [1, 2, 3],  
03     [4, 5, 6, 7],  
04     [8, 9]  
05 ]  
06  
07 for items in list_of_list:  
08     for item in items:  
09         print(items)
```



실행 결과

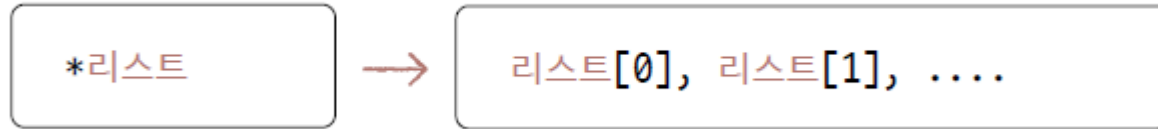
```
[1, 2, 3]  
[1, 2, 3]  
[1, 2, 3]  
[4, 5, 6, 7]  
[4, 5, 6, 7]  
[4, 5, 6, 7]  
[4, 5, 6, 7]  
[8, 9]  
[8, 9]
```


전개 연산자

- 전개 연산자

- 리스트 내용을 전개해서 입력 (리스트 앞에 * 기호를 사용)

1) 리스트 내부 2) 함수의 매개변수 위치에 사용



전개 연산자

- 리스트 내부에 사용

```
>>> a = [1, 2, 3, 4]
>>> b = [*a, *a]
>>> b
[1, 2, 3, 4, 1, 2, 3, 4]
```

append() 함수를 사용한 경우

```
>>> a = [1, 2, 3, 4]
>>> a.append(5)
>>> a
[1, 2, 3, 4, 5] → a 내용이 변경됩니다.
```

전개 연산자를 사용한 경우

```
>>> b = [1, 2, 3, 4]
>>> c = [*b, 5]
>>> b
[1, 2, 3, 4] → b 내용에는 어떠한 영향도 없습니다.
>>> c
[1, 2, 3, 4, 5] → 새로운 리스트 c가 만들어집니다.
```

전개 연산자

- 함수 매개변수 위치에 사용

```
>>> a = [1, 2, 3, 4]
>>> print(*a) → print(1, 2, 3, 4)처럼 동작합니다.
1 2 3 4
```

2. 딕셔너리와 반복문

시작하기 전에

- 딕셔너리(dictionary)
 - 키를 기반으로 값을 저장하는 것

```
{  
  키↓  값↓  
  "키A": 10,    # 문자열을 키로 사용하기  
  "키B": 20,  
  "키C": 30,  
  1:    40,      # 숫자를 키로 사용하기  
  False: 50      # 불을 키로 사용하기  
}
```

자료형	의미	가리키는 위치	선언 형식
리스트	인덱스를 기반으로 값을 저장	인덱스	변수 = []
딕셔너리	키를 기반으로 값을 저장	키	변수 = {}

딕셔너리 선언하기

- 딕셔너리 선언

- 중괄호로 선언하며 '키: 값' 형태를 쉼표로 연결해서 만들

```
변수 = {  
    키: 값,  
    키: 값,  
    ...  
    키: 값  
}
```

```
>>> dict_a = {  
...     "name": "어벤저스 엔드게임",  
...     "type": "히어로 무비"  
... }
```

딕셔너리의 요소에 접근하기

- 특정 키 값만 따로 출력하기
 - 딕셔너리 뒤에 대괄호 입력하고 그 내부에 키 입력

```
>>> dict_a  
{'name': '어벤저스 엔드게임', 'type': '히어로 무비'}
```

```
>>> dict_a["name"]  
'어벤저스 엔드게임'  
>>> dict_a["type"]  
'히어로 무비'
```

딕셔너리의 요소에 접근하기

- 딕셔너리 내부 값에 문자열, 숫자, 불 등 다양한 자료 넣기

```
>>> dict_b = {  
...     "director": ["안소니 루소", "조 루소"],  
...     "cast": ["아이언맨", "타노스", "토르", "닥터스트레인지", "헐크"]  
... }
```

```
>>> dict_b  
{'director': ['안소니 루소', '조 루소'], 'cast': ['아이언맨', '타노스', '토르', '닥터스트레인지', '헐크']}  
>>> dict_b["director"]  
['안소니 루소', '조 루소']
```


딕셔너리의 요소에 접근하기

구분	선언 형식	사용 예	틀린 예
리스트	<code>list_a = []</code>	<code>list_a[1]</code>	
딕셔너리	<code>dict_a = {}</code>	<code>dict_a["name"]</code>	<code>dict_a{"name"}</code>

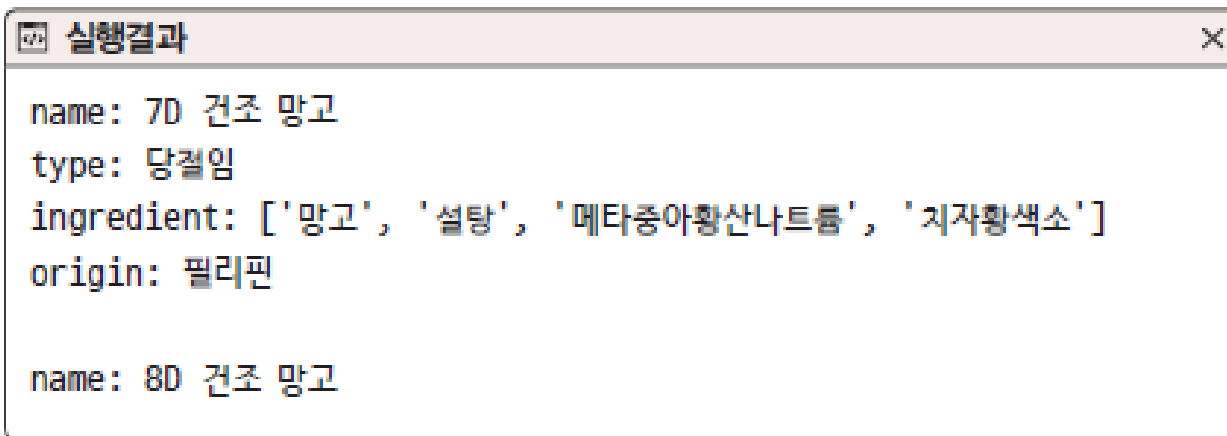
↓
처음 파이썬을 공부할 때
자주 실수하는 형태이므로
주의해 주세요.

- 예시 - 딕셔너리 요소에 접근하기

```
01  # 딕셔너리를 선언합니다.
02  dictionary = {
03      "name": "7D 건조 망고",
04      "type": "당절임",
05      "ingredient": ["망고", "설탕", "메타중아황산나트륨", "치자황색소"],
06      "origin": "필리핀"
07  }
08
```

딕셔너리의 요소에 접근하기

```
09  # 출력합니다.  
10  print("name:", dictionary["name"])  
11  print("type:", dictionary["type"])  
12  print("ingredient:", dictionary["ingredient"])  
13  print("origin:", dictionary["origin"])  
14  print()  
15  
16  # 값을 변경합니다.  
17  dictionary["name"] = "8D 건조 망고"  
18  print("name:", dictionary["name"])
```



```
실행결과  
name: 7D 건조 망고  
type: 당절임  
ingredient: ['망고', '설탕', '메타중아황산나트륨', '치자황색소']  
origin: 필리핀  
  
name: 8D 건조 망고
```

딕셔너리의 요소에 접근하기

- 리스트 안의 특정 값 출력하려는 경우

```
>>> dictionary["ingredient"]  
['망고', '설탕', '메타중아황산나트륨', '치자황색소']  
>>> dictionary["ingredient"][1]  
'설탕'
```

딕셔너리의 요소에 접근하기

- 딕셔너리의 문자열 키와 관련된 실수
- NameError 오류
 - name이라는 이름이 정의되지 않음

```
>>> dict_key = {  
...     name: "7D 건조 망고",  
...     type: "당절임"  
... }
```

 오류

```
Traceback (most recent call last):  
  File "<pyshell#5>", line 2, in <module>  
    name: "7D 건조 망고",  
NameError: name 'name' is not defined
```

딕셔너리의 요소에 접근하기

- name 이름을 변수로 만들어 해결

```
>>> name = "이름"
>>> dict_key = {
...     name: "7D 건조 망고",
...     type: "당절임"
... }
>>> dict_key
{'이름': '7D 건조 망고', <class 'type': '당절임'}
```

딕셔너리에 값 추가하기/제거하기

- 딕셔너리에 값 추가할 때는 키를 기반으로 값 입력

딕셔너리[새로운 키] = 새로운 값

- 슬라이드 #8, 9에서 만든 dictionary에 새로운 자료 추가

```
>>> dictionary["price"] = 5000
>>> dictionary
{'name': '8D 건조 망고', 'type': '당절임', 'ingredient': ['망고', '설탕', '메타중아황산나트륨', '치자황색소'], 'origin': '필리핀', 'price': 5000} → "price" 키가 추가되었습니다.
```

- 딕셔너리에 이미 존재하는 키 지정하고 값 넣으면 기존 값을 대치

```
>>> dictionary["name"] = "8D 건조 파인애플"
>>> dictionary
{'name': '8D 건조 파인애플', 'type': '당절임', 'ingredient': ['망고', '설탕', '메타중아황산나트륨', '치자황색소'], 'origin': '필리핀', 'price': 5000}
```

새로운 값으로 대치되었습니다.

딕셔너리에 값 추가하기/제거하기

- 딕셔너리 요소의 제거 : del 키워드

```
>>> del dictionary["ingredient"]  
>>> dictionary  
{'name': '8D 건조 파인애플', 'type': '당절임', 'origin': '필리핀', 'price': 5000}
```

딕셔너리에 값 추가하기/제거하기

- 예시 – 딕셔너리에 요소 추가하기

```
01  # 딕셔너리를 선언합니다.
02  dictionary = {}
03
04  # 요소 추가 전에 내용을 출력해 봅니다.
05  print("요소 추가 이전:", dictionary)
06
07  # 딕셔너리에 요소를 추가합니다.
08  dictionary["name"] = "새로운 이름"
09  dictionary["head"] = "새로운 정신"
10  dictionary["body"] = "새로운 몸"
11
12  # 출력합니다.
13  print("요소 추가 이후:", dictionary)
```

실행결과

요소 추가 이전: {}

요소 추가 이후: {'name': '새로운 이름', 'head': '새로운 정신', 'body': '새로운 몸'}

딕셔너리에 값 추가하기/제거하기

- 예시 – 딕셔너리에 요소 제거하기

```
01  # 딕셔너리를 선언합니다.
02  dictionary = {
03      "name": "7D 건조 망고",
04      "type": "당절임"
05  }
06
07  # 요소 제거 전에 내용을 출력해 봅니다.
08  print("요소 제거 이전:", dictionary)
09
10  # 딕셔너리의 요소를 제거합니다.
11  del dictionary["name"]
12  del dictionary["type"]
13
14  # 요소 제거 후에 내용을 출력해 봅니다.
15  print("요소 제거 이후:", dictionary)
```

실행결과

요소 제거 이전: {'name': '7D 건조 망고', 'type': '당절임'}
요소 제거 이후: {}

딕셔너리에 값 추가하기/제거하기

- KeyError 예외

- 딕셔너리에서 존재하지 않는 키에 접근할 경우

```
>>> dictionary = {}  
>>> dictionary["Key"]
```

 오류

```
Traceback (most recent call last):  
  File "<pyshell#7>", line 1, in <module>  
    dictionary["Key"]  
KeyError: 'Key'
```

딕셔너리에 값 추가하기/제거하기

- 값 제거할 경우도 같은 원리

```
>>> del dictionary["Key"]
Traceback (most recent call last):
  File "<pyshell#8>", line 1, in <module>
    del dictionary["Key"]
KeyError: 'Key' → KeyError가 발생합니다.
```

딕셔너리 내부에 키가 있는지 확인하기

- in 키워드

- 사용자로부터 접근하고자 하는 키 입력 받은 후 존재하는 경우에만 접근하여 값을 출력
- 예시 - 키가 존재하는지 확인하고 값에 접근하기

```
01  # 딕셔너리를 선언합니다.
02  dictionary = {
03      "name": "7D 건조 망고",
04      "type": "당절임",
05      "ingredient": ["망고", "설탕", "메타중아황산나트륨", "치자황색소"],
06      "origin": "필리핀"
07  }
```

딕셔너리 내부에 키가 있는지 확인하기

```
08
09  # 사용자로부터 입력을 받습니다.
10  key = input("> 접근하고자 하는 키: ")
11
12  # 출력합니다.
13  if key in dictionary:
14      print(dictionary[key])
15  else:
16      print("존재하지 않는 키에 접근하고 있습니다.")
```

 실행 결과 ✕

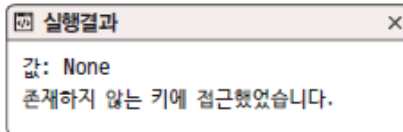
```
> 접근하고자 하는 키: name 
7D 건조 망고
> 접근하고자 하는 키: ㅇ ㅂ ㅇ 
존재하지 않는 키에 접근하고 있습니다.
```

딕셔너리 내부에 키가 있는지 확인하기

● get() 함수

- 딕셔너리의 키로 값을 추출 - 존재하지 않는 키에 접근할 경우 None 출력
- 예시 - 키가 존재하지 않을 때 None을 출력하는지 확인하기

```
01 # 딕셔너리를 선언합니다.
02 dictionary = {
03     "name": "7D 건조 망고",
04     "type": "당절임",
05     "ingredient": ["망고", "설탕", "메타중아황산나트륨", "치자황색소"],
06     "origin": "필리핀"
07 }
08
09 # 존재하지 않는 키에 접근해 봅니다.
10 value = dictionary.get("존재하지 않는 키")
11 print("값:", value)
12
13 # None 확인 방법
14 if value == None: → None과 같은지 확인만 하면 됩니다.
15     print("존재하지 않는 키에 접근했습니다.")
```



for 반복문 : 딕셔너리와 함께 사용하기

- for 반복문과 딕셔너리의 조합

```
for 키 변수 in 딕셔너리:
```

```
    코드
```

for 반복문 : 딕셔너리와 함께 사용하기

```
01  # 딕셔너리를 선언합니다.
02  dictionary = {
03      "name": "7D 건조 망고",
04      "type": "당절임",
05      "ingredient": ["망고", "설탕", "메타중아황산나트륨", "치자황색소"],
06      "origin": "필리핀"
07  }
08
09  # for 반복문을 사용합니다.
10  for key in dictionary:
11      # 출력합니다.
12      print(key, ":", dictionary[key])
```

실행결과

```
name : 7D 건조 망고
type : 당절임
ingredient : ['망고', '설탕', '메타중아황산나트륨', '치자황색소']
origin : 필리핀
```


3. 범위 자료형과 반복문

시작하기 전에

- 범위(range)
 - 특정 횟수만큼 반복해서 돌리고 싶을 때 for 반복문과 조합하여 사용

범위

- 매개변수에 숫자를 한 개 넣는 방법

- 0부터 A-1까지의 정수로 범위 만들

`range(A)` → A는 숫자

- 매개변수에 숫자를 두 개 넣는 방법

- A부터 B-1까지의 정수로 범위 만들

`range(A, B)` → A와 B는 숫자

- 매개변수에 숫자를 세 개 넣는 방법

- A부터 B-1까지의 정수로 범위 만들되 앞뒤의 숫자가 C만큼의 차이 가짐

`range(A, B, C)` → A, B, C는 숫자

범위

- 예시
 - 매개변수에 숫자 한 개 넣은 범위

```
>>> a = range(5)
```

```
>>> a  
range(0, 5)
```

```
>>> list(range(5))  
[0, 1, 2, 3, 4]
```

범위

- 매개변수에 숫자 두 개 넣은 범위

```
>>> list(range(0, 5)) → 0부터 (5-1)까지의 정수로 범위를 만듭니다.  
[0, 1, 2, 3, 4]  
>>> list(range(5, 10)) → 5부터 (10-1)까지의 정수로 범위를 만듭니다.  
[5, 6, 7, 8, 9]
```

- 매개변수에 숫자 세 개 넣은 범위

```
>>> list(range(0, 10, 2)) → 0부터 2씩 증가하면서 (10-1)까지의 정수로 범위를 만듭니다.  
[0, 2, 4, 6, 8]  
>>> list(range(0, 10, 3)) → 0부터 3씩 증가하면서 (10-1)까지의 정수로 범위를 만듭니다.  
[0, 3, 6, 9]
```

범위

- 범위 만들 때 매개변수 내부에 수식 사용하는 경우
 - 코드 특정 부분의 강조

```
>>> a = range(0, 10 + 1)
>>> list(a)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

- 예시 - 나누기 연산자 사용

```
>>> n = 10
>>> a = range(0, n / 2) → 매개변수로 나눗셈을 사용한 경우 오류가 발생합니다.
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
TypeError: 'float' object cannot be interpreted as an integer
```

=> TypeError 발생 - range() 함수의 매개변수로는 반드시 '정수'를 입력해야 하기 때문

범위

- 정수 나누기 연산자

```
>>> a = range(0, int(n / 2)) → 실수를 정수로 바꾸는 방법보다
>>> list(a)
[0, 1, 2, 3, 4]

>>> a = range(0, n // 2) → 정수 나누기 연산자를 많이 사용합니다!
>>> list(a)
[0, 1, 2, 3, 4]
```

for 반복문: 범위와 함께 사용하기

- for 반복문과 범위의 조합

```
for 숫자 변수 in 범위:
```

코드

```
01 # for 반복문과 범위를 함께 조합해서 사용합니다.  
02 for i in range(5):  
03     print(str(i) + "= 반복 변수")  
04 print()  
05  
06 for i in range(5, 10):  
07     print(str(i) + "= 반복 변수")  
08 print()  
09  
10 for i in range(0, 10, 3):  
11     print(str(i) + "= 반복 변수")  
12 print()
```

실행결과

0	=	반복 변수
1	=	반복 변수
2	=	반복 변수
3	=	반복 변수
4	=	반복 변수
5	=	반복 변수
6	=	반복 변수
7	=	반복 변수
8	=	반복 변수
9	=	반복 변수
0	=	반복 변수
3	=	반복 변수
6	=	반복 변수
9	=	반복 변수

for 반복문 : 리스트와 범위 조합하기

- 몇 번째 반복인지를 알아야 하는 경우

```
# 리스트를 선언합니다.  
array = [273, 32, 103, 57, 52]  
# 리스트에 반복문을 적용합니다.  
for element in array:  
    # 출력합니다.  
    print(element)
```

↓
현재 무엇을 출력하고 있는지 보다, 몇 번째 출력인지를 알아야 하는 경우가 있습니다.

```
01 # 리스트를 선언합니다.  
02 array = [273, 32, 103, 57, 52]  
03  
04 # 리스트에 반복문을 적용합니다.  
05 for i in range(len(array)):  
06     # 출력합니다.  
07     print("{}번째 반복: {}".format(i, array[i]))
```

실행결과

0번째 반복:	273
1번째 반복:	32
2번째 반복:	103
3번째 반복:	57
4번째 반복:	52

for 반복문: 반대로 반복하기

- 역반복문

- 큰 숫자에서 작은 숫자로 반복문 적용
- `range()` 함수의 매개변수 세 개 사용하는 방법
- 예시 - 반대로 반복하기(1)

```
01  # 역반복문
02  for i in range(4, 0 - 1, -1):
03      # 출력합니다.
04      print("현재 반복 변수: {}".format(i))
```

실행결과

현재 반복 변수: 4
현재 반복 변수: 3
현재 반복 변수: 2
현재 반복 변수: 1
현재 반복 변수: 0

for 반복문: 반대로 반복하기

- reversed() 함수 사용하는 방법
- 예시 - 반대로 반복하기(2)

```
01  # 역반복문
02  for i in reversed(range(5)):
03      # 출력합니다.
04      print("현재 반복 변수: {}".format(i))
```



실행결과

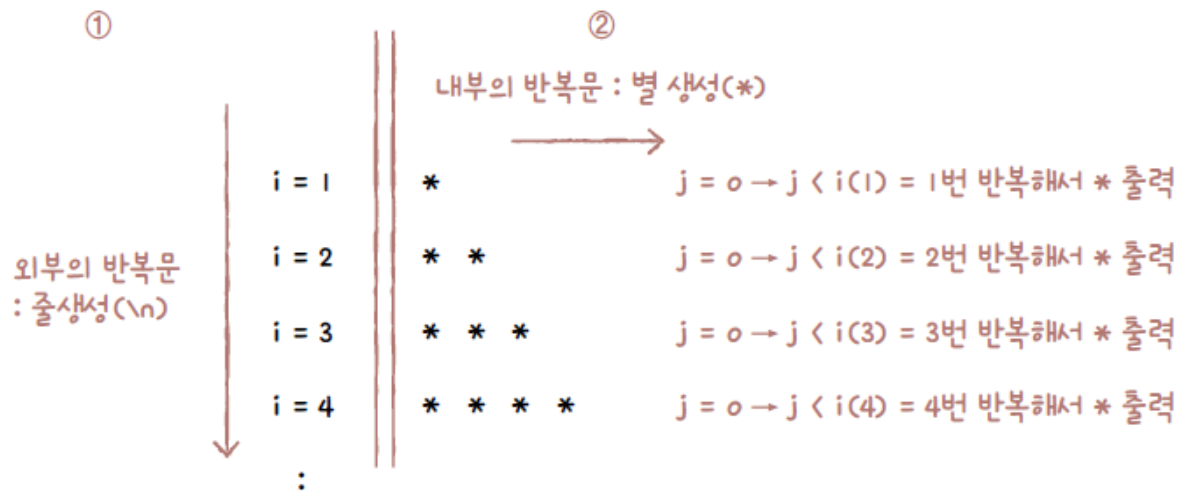
현재 반복 변수: 4
현재 반복 변수: 3
현재 반복 변수: 2
현재 반복 변수: 1
현재 반복 변수: 0

중첩 반복문으로 피라미드 만들기(누적 예제)

- 예시 - 반복문으로 피라미드 만들기(1)

```
01 output = ""
02
03 for i in range(1, 10):
04     for j in range(0, i):
05         output += "*"
06     output += "\n"
07
08 print(output)
```

```
*
**
***
****
*****
*****
*****
*****
*****
```



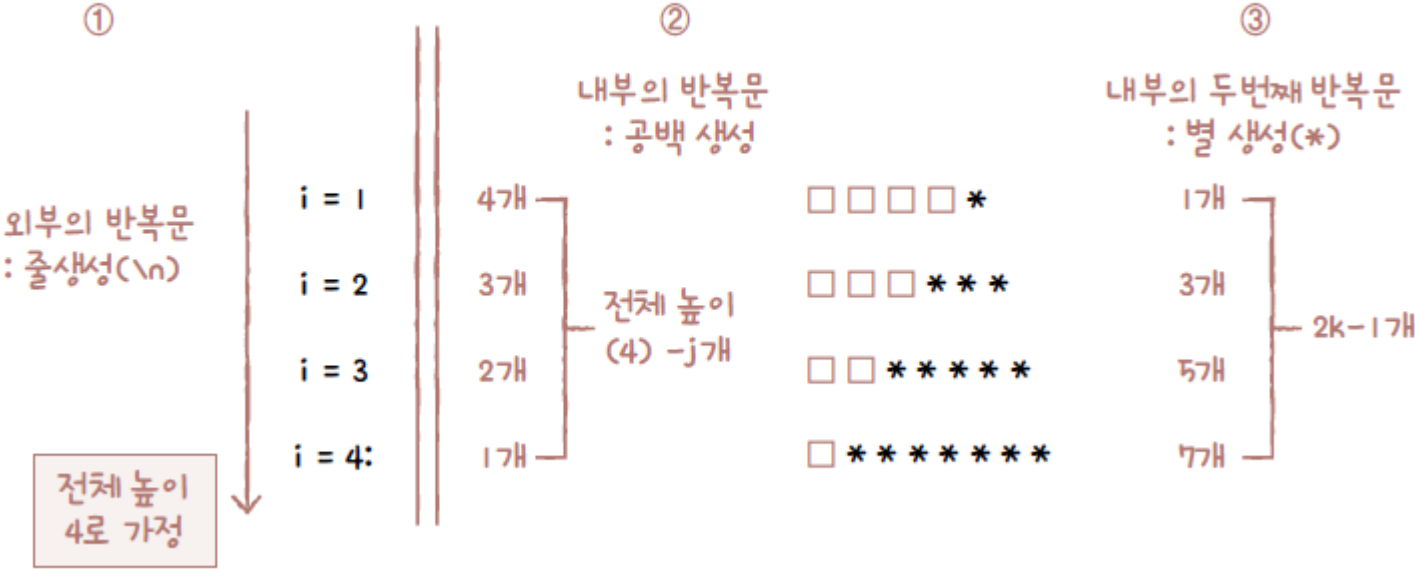
중첩 반복문으로 피라미드 만들기(누적 예제)

- 예시 - 반복문으로 피라미드 만들기(2)

```
01 output = ""
02
03 for i in range(1, 15):
04     for j in range(14, i, -1):
05         output += ' '
06     for k in range(0, 2 * i - 1):
07         output += '*'
08     output += '\n'
09
10 print(output)
```

```
      *
     ***
    *****
   *********
  ***********
 *****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

중첩 반복문으로 피라미드 만들기(누적 예제)



while 반복문

- while 반복문

- 리스트 또는 딕셔너리 내부의 요소를 특정 횟수만큼 반복
- 예시 – 무한 반복

```
while 불 표현식:  
    문장
```

```
01  # while 반복문을 사용합니다.  
02  while True:  
03      # "."을 출력합니다.  
04      # 기본적으로 end가 "\n"이라 줄바꿈이 일어나는데  
05      # 빈 문자열 ""로 바꿔서 줄바꿈이 일어나지 않게 합니다.  
06      print(".", end="")
```

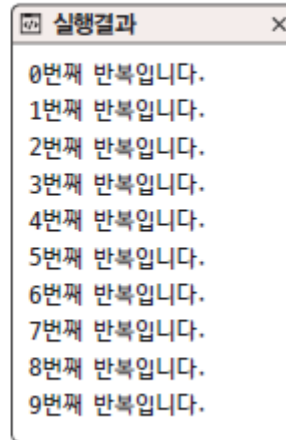
실행결과

```
.....  
.....  
.....  
.....
```

while 반복문 : for 반복문처럼 사용하기

- 예시 – while 반복문을 for 반복문처럼 사용하기

```
01  # 반복 변수를 기반으로 반복하기
02  i = 0
03  while i < 10:
04      print("{}번째 반복입니다.".format(i))
05      i += 1
```



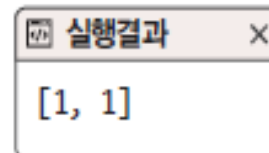
실행결과

0번째 반복입니다.
1번째 반복입니다.
2번째 반복입니다.
3번째 반복입니다.
4번째 반복입니다.
5번째 반복입니다.
6번째 반복입니다.
7번째 반복입니다.
8번째 반복입니다.
9번째 반복입니다.

while 반복문: 상태를 기반으로 반복하기

- 리스트 내부에서 해당하는 값을 여러 개 제거
 - while 반복문의 조건을 '리스트 내부에 요소가 있는 동안'으로 지정
 - 예시 - 해당하는 값 모두 제거하기

```
01  # 변수를 선언합니다.  
02  list_test = [1, 2, 1, 2]  
03  value = 2  
04  
05  # list_test 내부에 value가 있다면 반복  
06  while value in list_test:  
07      list_test.remove(value)  
08  
09  # 출력합니다.  
10  print(list_test)
```



while 반복문: 시간을 기반으로 반복하기

- 예시 - 유닉스 타임 구하기

- 시간 관련된 기능 가져오기

```
>>> import time
```

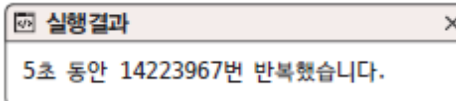
- 유닉스 타임

```
>>> time.time()  
1557241486.6654928
```

while 반복문: 시간을 기반으로 반복하기

- 유닉스 타임과 while 반복문을 조합
 - 예시 - 5초 동안 반복하기

```
01  # 시간과 관련된 기능을 가져옵니다.
02  import time
03
04  # 변수를 선언합니다.
05  number = 0
06
07  # 5초 동안 반복합니다.
08  target_tick = time.time() + 5
09  while time.time() < target_tick:
10      number += 1
11
12  # 출력합니다.
13  print("5초 동안 {}번 반복했습니다.".format(number))
```



실행결과

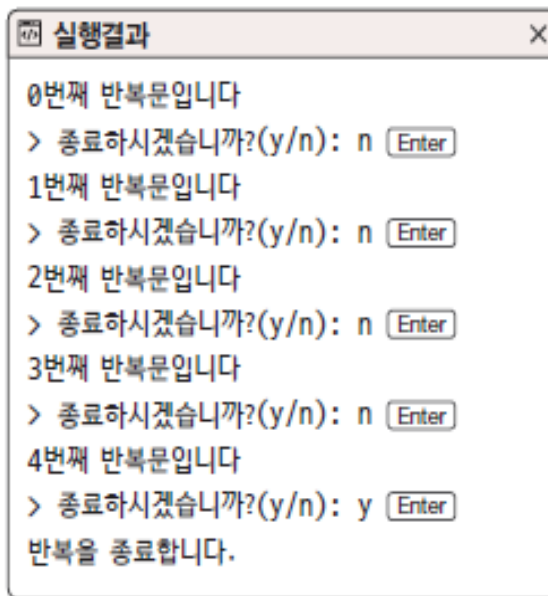
5초 동안 14223967번 반복했습니다.

while 반복문: break 키워드/continue 키워드

- break 키워드

- 반복문 벗어날 때 사용하는 키워드

```
01  # 변수를 선언합니다.  
02  i = 0  
03  
04  # 무한 반복합니다.  
05  while True:  
06      # 몇 번째 반복인지 출력합니다.  
07      print("{}번째 반복문입니다.".format(i))  
08      i = i + 1  
09      # 반복을 종료합니다.  
10      input_text = input("> 종료하시겠습니까?(y): ")  
11      if input_text in ["y", "Y"]:  
12          print("반복을 종료합니다.")  
13          break
```



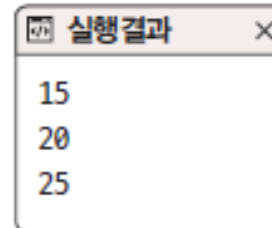
```
실행결과  
0번째 반복문입니다  
> 종료하시겠습니까?(y/n): n Enter  
1번째 반복문입니다  
> 종료하시겠습니까?(y/n): n Enter  
2번째 반복문입니다  
> 종료하시겠습니까?(y/n): n Enter  
3번째 반복문입니다  
> 종료하시겠습니까?(y/n): n Enter  
4번째 반복문입니다  
> 종료하시겠습니까?(y/n): y Enter  
반복을 종료합니다.
```

while 반복문: break 키워드/continue 키워드

- continue 키워드

- 현재 반복을 생략하고 다음 반복으로 넘어감

```
01  # 변수를 선언합니다.  
02  numbers = [5, 15, 6, 20, 7, 25]  
03  
04  # 반복을 돌립니다.  
05  for number in numbers:  
06      # number가 10보다 작으면 다음 반복으로 넘어갑니다.  
07      if number < 10:  
08          continue  
09      # 출력합니다.  
10      print(number)
```



while 반복문: break 키워드/continue 키워드

- if else 구문 사용도 가능한 경우이나, continue 키워드 사용하면 이후 처리의 들여쓰기를 하나 줄일 수 있음

continue 키워드를 사용하지 않은 경우

```
# 반복을 돌립니다.  
for number in numbers:  
    # 반복 대상을 한정합니다.  
    if number >= 10:  
        # 문장  
        # 문장  
        # 문장  
        # 문장  
        # 문장
```

continue 키워드를 사용한 경우

```
# 반복을 돌립니다.  
for number in numbers:  
    # 반복 대상에서 제외해버립니다.  
    if number < 10:  
        continue  
    # 문장  
    # 문장  
    # 문장  
    # 문장  
    # 문장
```

4. 문자열, 리스트, 딕셔너리와 관련된 기본 함수

시작하기 전에

- 파이썬만의 고유한 기능들

- 리스트에 적용할 수 있는 기본 함수 : `min()`, `max()`, `sum()`
- 리스트 뒤집기 : `reversed()`
- 현재 인덱스가 몇 번째인지 확인하기 : `enumerate()`
- 딕셔너리로 쉽게 반복문 작성하기 : `item()`
- 리스트 안에 `for`문 사용하기 : 리스트 내포

리스트에 적용할 수 있는 기본 함수

- min(), max(), sum() 함수

- 리스트를 매개변수로 넣어 사용하는 기본 함수들

함수	설명
min()	리스트 내부에서 최솟값을 찾습니다.
max()	리스트 내부에서 최댓값을 찾습니다.
sum()	리스트 내부에서 값을 모두 더합니다.

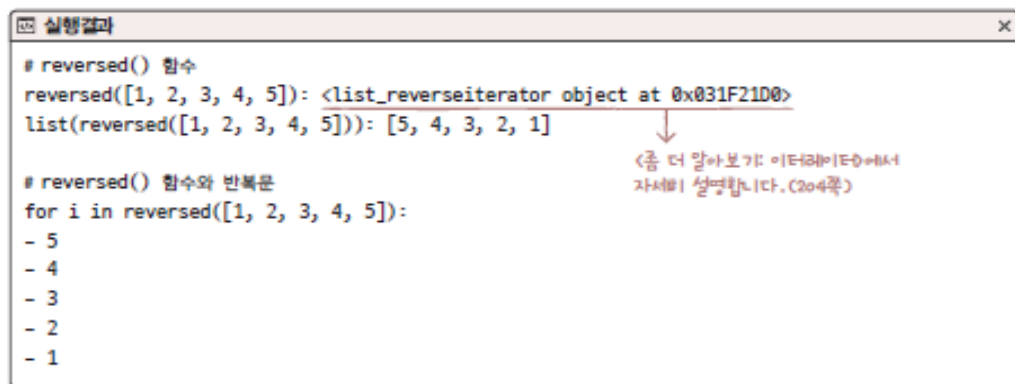
```
>>> numbers = [103, 52, 273, 32, 77]
>>> min(numbers)           → 리스트 내부에서 최솟값을 찾습니다.
32
>>> max(numbers)           → 리스트 내부에서 최댓값을 찾습니다.
273
>>> sum(numbers)           → 리스트 내부에서 값을 모두 더합니다.
537
```

reversed() 함수로 리스트 뒤집기

- reversed() 함수

- 리스트에서 요소 순서 뒤집기

```
01  # 리스트를 선언하고 뒤집습니다.
02  list_a = [1, 2, 3, 4, 5]
03  list_reversed = reversed(list_a)
04
05  # 출력합니다.
06  print("# reversed() 함수")
07  print("reversed([1, 2, 3, 4, 5]):", list_reversed)
08  print("list(reversed([1, 2, 3, 4, 5])):", list(list_reversed))
09  print()
10
11  # 반복문을 적용해 봅니다.
12  print("# reversed() 함수와 반복문")
13  print("for i in reversed([1, 2, 3, 4, 5]):")
14  for i in reversed(list_a):
15      print("-", i)
```



The screenshot shows a Python interpreter window titled "실행결과" (Execution Result). It displays the output of the code from the previous block. The first part shows the result of the reversed() function, which is a list_reverseiterator object. The second part shows the output of a loop that iterates over the reversed list, printing the elements in reverse order: 5, 4, 3, 2, 1. A red arrow points from the text "<좀 더 알아보기: 이터레이터에 대해서 자세히 설명합니다. (204쪽)>" to the list_reverseiterator object in the first output line.

```
# reversed() 함수
reversed([1, 2, 3, 4, 5]): <list_reverseiterator object at 0x031F21D0>
list(reversed([1, 2, 3, 4, 5])): [5, 4, 3, 2, 1]

# reversed() 함수와 반복문
for i in reversed([1, 2, 3, 4, 5]):
- 5
- 4
- 3
- 2
- 1
```

reversed() 함수로 리스트 뒤집기

- reversed() 함수와 반복문 조합할 때는 함수 결과 여러 번 활용하지 않고 for 구문 내부에 reversed() 함수 곧바로 넣어서 사용

- 잘못된 예

```
temp = reversed([1, 2, 3, 4, 5, 6])

for i in temp:
    print("첫 번째 반복문: {}".format(i))

for i in temp:
    print("두 번째 반복문: {}".format(i))
```

```
첫 번째 반복문: 6
첫 번째 반복문: 5
첫 번째 반복문: 4
첫 번째 반복문: 3
첫 번째 반복문: 2
첫 번째 반복문: 1
```

reversed() 함수로 리스트 뒤집기

- 바른 예

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
for i in reversed(numbers):  
    print("첫 번째 반복문: {}".format(i))
```

```
for i in reversed(numbers):  
    print("두 번째 반복문: {}".format(i))
```

→ 필요한 시점에 reversed() 함수를 사용합니다.

enumerate() 함수와 반복문 조합하기

- enumerate() 함수

- 리스트 요소 반복할 때 현재 인덱스가 몇 번째인지 확인
- 예시

```
example_list = ["요소A", "요소B", "요소C"]
```

0번째 요소는 요소A입니다.

1번째 요소는 요소B입니다.

2번째 요소는 요소C입니다.

enumerate() 함수와 반복문 조합하기

- 방법 1

```
example_list = ["요소A", "요소B", "요소C"]  
i = 0  
for item in example_list:  
    print("{}번째 요소는 {}".format(i, item))  
    i += 1
```

- 방법 2

```
example_list = ["요소A", "요소B", "요소C"]  
for i in range(len(example_list)):  
    print("{}번째 요소는 {}".format(i, example_list[i]))
```

enumerate() 함수와 반복문 조합하기

- 예시 – enumerate() 함수와 리스트

```
01  # 변수를 선언합니다.
02  example_list = ["요소A", "요소B", "요소C"]
03
04  # 그냥 출력합니다.
05  print("# 단순 출력")
06  print(example_list)
07  print()
08
09  # enumerate() 함수를 적용해 출력합니다.
10  print("# enumerate() 함수 적용 출력")
11  print(enumerate(example_list))
12  print()
13
14  # list() 함수로 강제 변환해 출력합니다.
15  print("# list() 함수로 강제 변환 출력")
16  print(list(enumerate(example_list)))
```

enumerate() 함수와 반복문 조합하기

```
17 print()
18
19 # for 반복문과 enumerate() 함수 조합해서 사용하기
20 print("# 반복문과 조합하기")
21 for i, value in enumerate(example_list):
22     print("{}번째 요소는 {}".format(i, value))
```

enumerate() 함수를 사용하면
반복 변수를 이런 형태로 넣을 수
있습니다.

실행결과

```
# 단순 출력
['요소A', '요소B', '요소C']

# enumerate() 함수 적용 출력
<enumerate object at 0x02A43CB0>

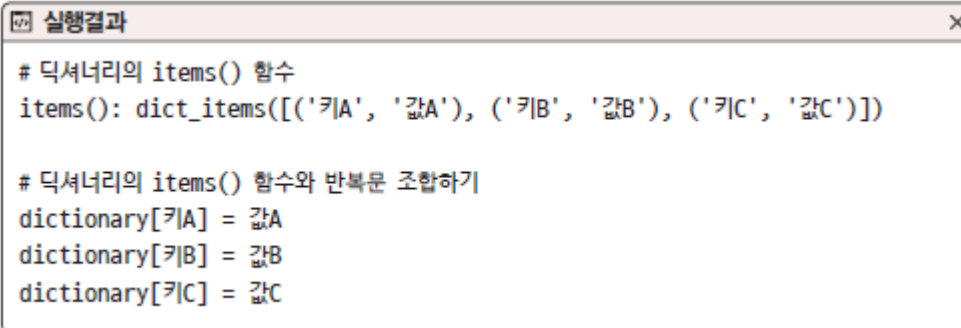
# list() 함수로 강제 변환 출력
[(0, '요소A'), (1, '요소B'), (2, '요소C')]

# 반복문과 조합하기
0번째 요소는 요소A입니다.
1번째 요소는 요소B입니다.
2번째 요소는 요소C입니다.
```


딕셔너리의 items() 함수와 반복문 조합하기

- 딕셔너리와 items() 함수 함께 사용하면 키와 값을 조합하여 쉽게 반복문 작성할 수 있음

```
01  # 변수를 선언합니다.
02  example_dictionary = {
03      "키A": "값A",
04      "키B": "값B",
05      "키C": "값C",
06  }
07
08  # 딕셔너리의 items() 함수 결과 출력하기
09  print("# 딕셔너리의 items() 함수")
10  print("items():", example_dictionary.items())
11  print()
12
13  # for 반복문과 items() 함수 조합해서 사용하기
14  print("# 딕셔너리의 items() 함수와 반복문 조합하기")
15
16  for key, element in example_dictionary.items():
17      print("dictionary[{}] = {}".format(key, element))
```



```
실행결과
# 딕셔너리의 items() 함수
items(): dict_items([('키A', '값A'), ('키B', '값B'), ('키C', '값C')])

# 딕셔너리의 items() 함수와 반복문 조합하기
dictionary[키A] = 값A
dictionary[키B] = 값B
dictionary[키C] = 값C
```

리스트 내포

- 반복문 사용하여 리스트 재조합하는 경우
 - 예시 – 반복문을 사용한 리스트 생성

```
01  # 변수를 선언합니다.  
02  array = []  
03  
04  # 반복문을 적용합니다.  
05  for i in range(0, 20, 2):  
06      array.append(i * i)  
07  # 출력합니다.  
08  print(array)
```

실행결과

[0, 4, 16, 36, 64, 100, 144, 196, 256, 324]

리스트 내포

- 예시 – 리스트 안에 for문 사용하기

```
01  # 리스트를 선언합니다.  
02  array = [i * i for i in range(0, 20, 2)]  
03           ↓  
           최종 결과를 앞에작성합니다.  
04  # 출력합니다.  
05  print(array)
```

- 리스트 내포 (list comprehension)

리스트 이름 = [표현식 for 반복자 in 반복할 수 있는 것]

리스트 내포

- 예시 - 조건을 활용한 리스트 내포

리스트 이름 = [표현식 for 반복자 in 반복할 수 있는 것 if 조건문]

```
01  # 리스트를 선언합니다.  
02  array = ["사과", "자두", "초콜릿", "바나나", "체리"]  
03  output = [fruit for fruit in array if fruit != "초콜릿"]  
04  
05  # 출력합니다.  
06  print(output)
```

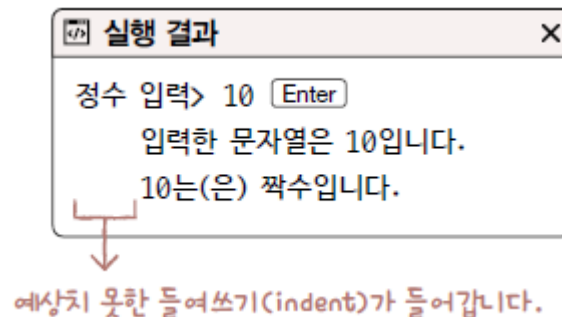
실행결과

['사과', '자두', '바나나', '체리']

구문 내부에 여러 줄 문자열을 사용했을 때 문제점

- 예시 – if 조건문과 여러 줄 문자열(1)

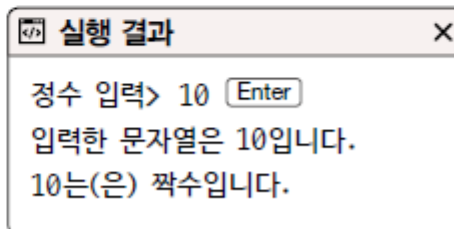
```
01  # 변수를 선언합니다.
02  number = int(input("정수 입력> "))
03
04  # if 조건문으로 홀수 짝수를 구분합니다.
05  if number % 2 == 0:
06      print("""\
07          입력한 문자열은 {}입니다.
08          {}는(은) 짝수입니다.""").format(number, number))
09  else:
10      print("""\
11          입력한 문자열은 {}입니다.
12          {}는(은) 홀수입니다.""").format(number, number))
```



구문 내부에 여러 줄 문자열을 사용했을 때 문제점

- 예시 - if 조건문과 여러 줄 문자열(2)

```
01  # 변수를 선언합니다.  
02  number = int(input("정수 입력> "))  
03  
04  # if 조건문으로 홀수 짝수를 구분합니다.  
05  if number % 2 == 0:  
06      print("""입력한 문자열은 {}입니다.  
07      {}는(은) 짝수입니다.""").format(number, number))  
08  else:  
09      print("""입력한 문자열은 {}입니다.  
10      {}는(은) 홀수입니다.""").format(number, number))
```



구문 내부에 여러 줄 문자열을 사용했을 때 문제점

- 예시 - if 조건문과 긴 문자열

```
01  # 변수를 선언합니다.  
02  number = int(input("정수 입력> "))  
03  
04  # if 조건문으로 홀수 짝수를 구분합니다.  
05  if number % 2 == 0:  
06      print("입력한 문자열은 {}입니다.\n{}는(은) 짝수입니다.".format(number, number))  
07  else:  
08      print("입력한 문자열은 {}입니다.\n{}는(은) 홀수입니다.".format(number, number))
```

구문 내부에 여러 줄 문자열을 사용했을 때 문제점

- 예시 - 괄호로 문자열 연결하기

```
01 # 변수를 선언합니다.  
02 test = (  
03     "이렇게 입력해도 "  
04     "하나의 문자열로 연결되어 "  
05     "생성됩니다."  
06 )  
07  
08 # 출력합니다.  
09 print("test:", test)  
10 print("type(test):", type(test))
```

실행 결과

test: 이렇게 입력해도 하나의 문자열로 연결되어 생성됩니다.
type(test): <class 'str'>

↓
문자열 자료형입니다.

구문 내부에 여러 줄 문자열을 사용했을 때 문제점

- 예시 - 여러 줄 문자열과 if 구문을 조합했을 때의 문제 해결(1)

```
01  # 변수를 선언합니다.
02  number = int(input("정수 입력> "))
03
04  # if 조건문으로 홀수 짝수를 구분합니다.
05  if number % 2 == 0:
06      print((
07          "입력한 문자열은 {}입니다.\n"
08          "{}는(은) 짝수입니다."
09      ).format(number, number))
10  else:
11      print((
12          "입력한 문자열은 {}입니다.\n"
13          "{}는(은) 홀수입니다."
14      ).format(number, number))
```

구문 내부에 여러 줄 문자열을 사용했을 때 문제점

- 문자열의 join() 함수 - 리스트의 요소를 문자열로 연결

문자열.join(문자열로 구성된 리스트)

```
>>> print(":".join(["1", "2", "3", "4", "5"]))  
1::2::3::4::5
```

구문 내부에 여러 줄 문자열을 사용했을 때 문제점

- 예시 - 여러 줄 문자열과 if 구문을 조합했을 때의 문제 해결(2)

```
01  # 변수를 선언합니다.
02  number = int(input("정수 입력> "))
03
04  # if 조건문으로 홀수 짝수를 구분합니다.
05  if number % 2 == 0:
06      print("\n".join([
07          "입력한 문자열은 {}입니다.",
08          "{}는(은) 짝수입니다."
09      ]).format(number, number))
10  else:
11      print("\n".join([
12          "입력한 문자열은 {}입니다.",
13          "{}는(은) 홀수입니다."
14      ]).format(number, number))
```

이터레이터

- 이터러블(iterable)

- 내부에 있는 요소들을 차례차례 꺼낼 수 있는 객체
- 리스트, 딕셔너리, 문자열, 튜플(5장 참고) 등

```
for 반복자 in 반복할 수 있는 것
```

- 이터레이터(iterator)

- 이터러블 중 next() 함수를 적용해 하나하나 꺼낼 수 있는 요소

이터레이터

- 예시 - reversed() 함수와 이터레이터

```
01  # 변수를 선언합니다.  
02  numbers = [1, 2, 3, 4, 5, 6]  
03  r_num = reversed(numbers)  
04  
05  # reversed_numbers를 출력합니다.  
06  print("reversed_numbers :", r_num)  
07  print(next(r_num))  
08  print(next(r_num))  
09  print(next(r_num))  
10  print(next(r_num))  
11  print(next(r_num))
```

실행 결과

```
reversed_numbers: <list_reverseiterator object at 0x034D21D0>  
6  
5  
4  
3  
2
```